

Bases de Datos

Clase 14: Índices

Índices

Herramientas que optimiza el acceso a los datos para una consulta o conjunto de consultas en particular

Idealmente un índice debe caber en RAM

Índices

```
CREATE INDEX <nombre índice> ON table  
<atributo>
```

Las llaves primarias están indexadas por defecto

Índices

A un índice yo le indico que quiero las tuplas con cierto valor en un atributo (o con un valor en cierto rango)

El índice puede devolver las tuplas, o punteros que me indican donde encontrarlas

Esto se logra hacer de manera rápida

Consultas a Optimizar

Consultas por valor

```
SELECT *  
FROM Table  
WHERE Table.value = 'value'
```

Consultas por rango

```
SELECT *  
FROM Table  
WHERE Table.value >= 'value'
```

Índices

Flujo

Supongamos que tengo la tabla:

```
Usuarios(uid: int PRIMARY KEY,  
         nombre: text,  
         edad: int)
```

en donde tenemos indexada la tabla por la primary key, que es el atributo `uid`

Índices

Flujo

El índice normalmente es una colección de archivos que puede ir completo en memoria

Si hacemos la consulta:

```
SELECT * FROM Usuarios WHERE uid = 104
```

esta se resolverá con el índice, así evitaremos recorrer toda la tabla

Índices

Flujo

El índice encontrará rápidamente la tupla que cumple con la condición

El índice tiene dos opciones para responder:

- Nos va a devolver la tupla que buscamos
- Nos va a devolver un puntero hacia la dirección de la página en el disco duro donde encontraremos la tupla

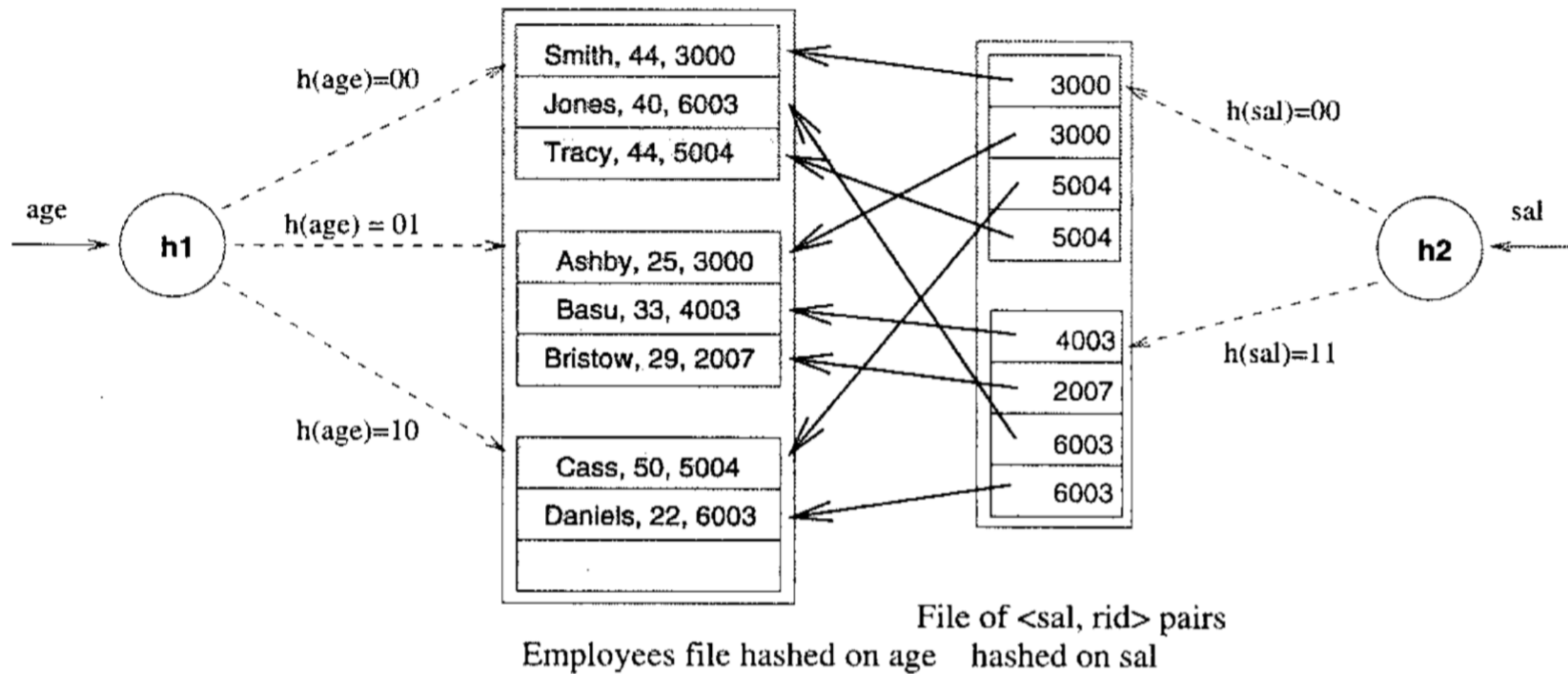
Índices

Definiciones

Search Key: es el parámetro con el que busco algo en mi índice (por ejemplo, el `uid` de una tabla de usuarios)

Data Entry: la tupla que nos retorna el índice después de preguntar por una Search Key en particular

Índices



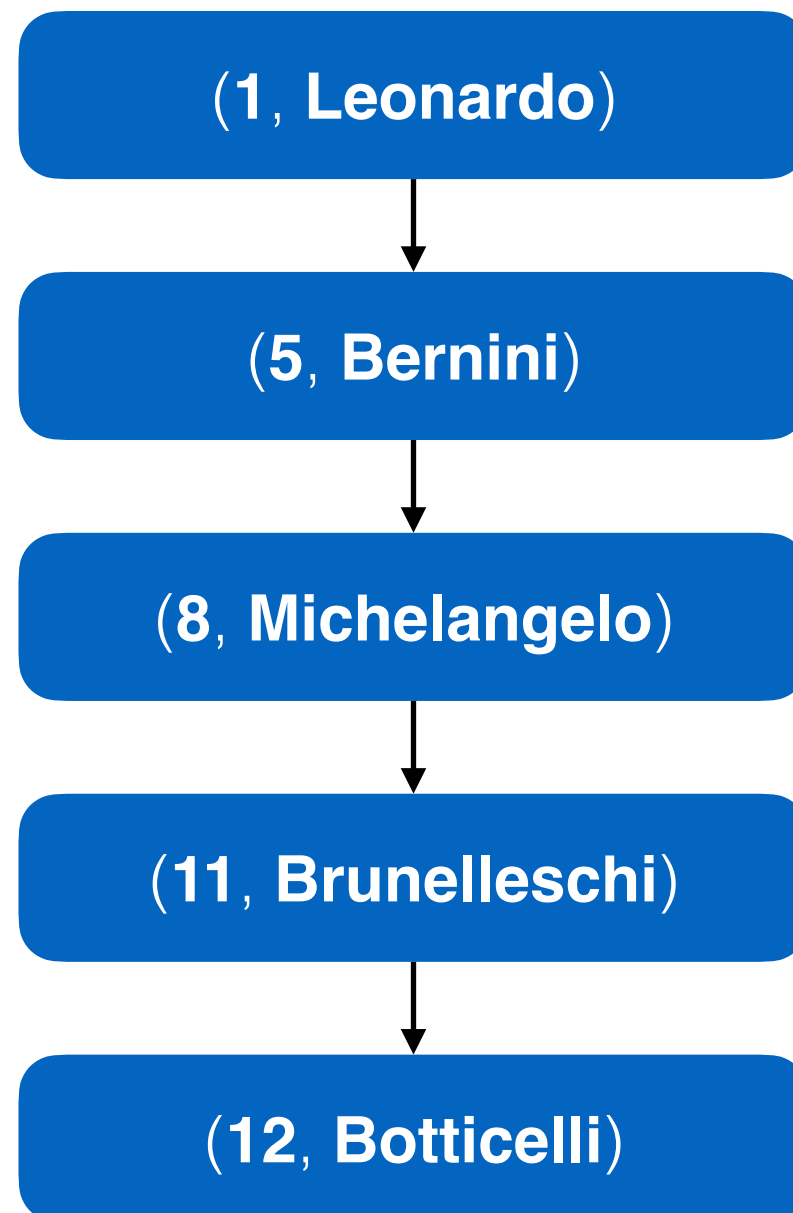
Repaso (o spoiler) de EDD

Queremos guardar una lista de tuplas en Python:

```
t = [  
    (1, Leonardo, ...),  
    (5, Bernini, ...),  
    (8, Michelangelo, ...),  
    (11, Brunelleschi, ...),  
    (12, Botticelli, ...)  
]
```

Repaso (o spoiler) de EDD

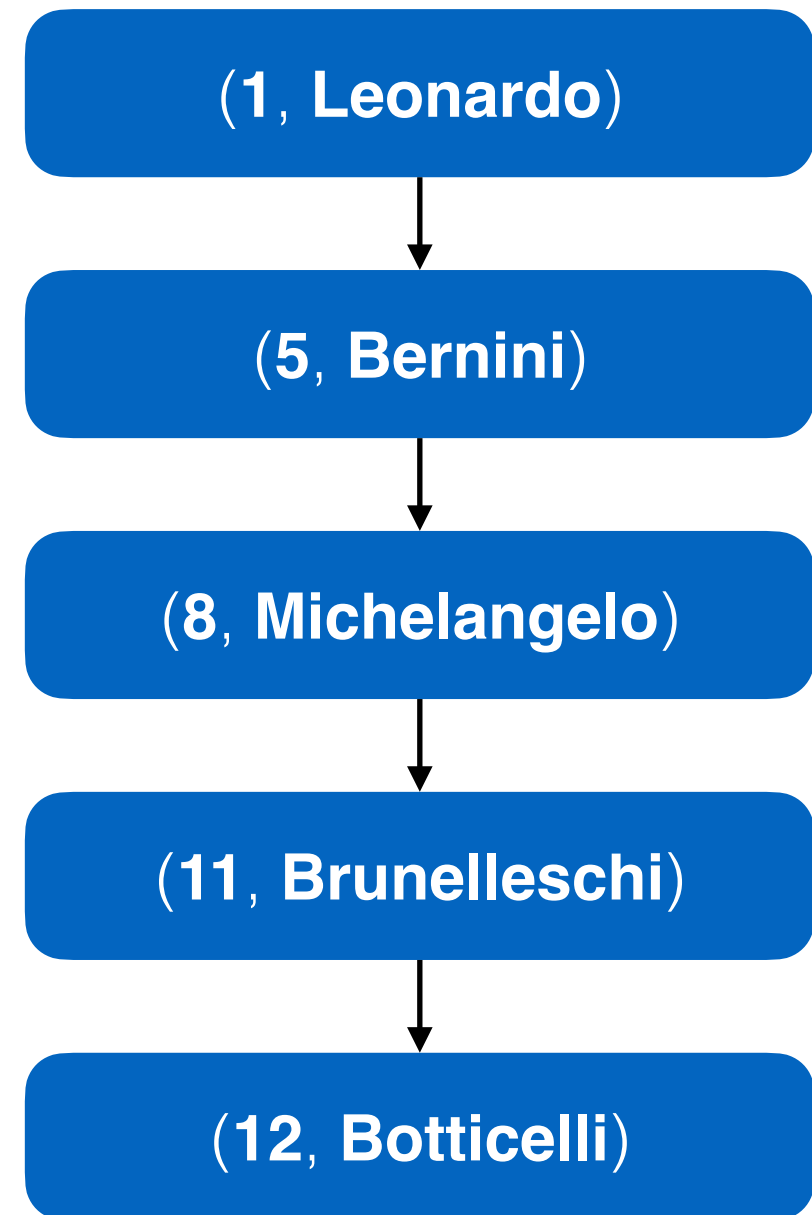
Lista ligada (versión básica)



Repaso (o spoiler) de EDD

Lista ligada (versión básica)

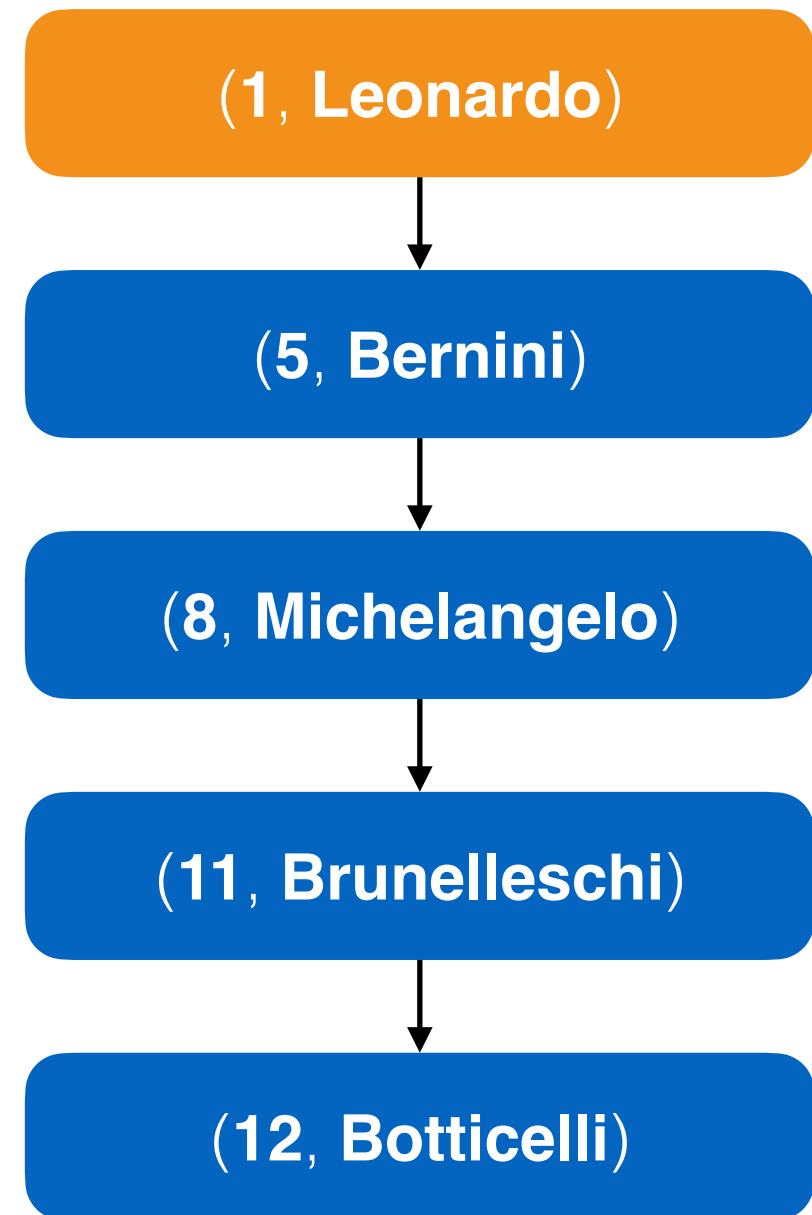
Supongamos que queremos el artista con identificador 11



Repaso (o spoiler) de EDD

Lista ligada (versión básica)

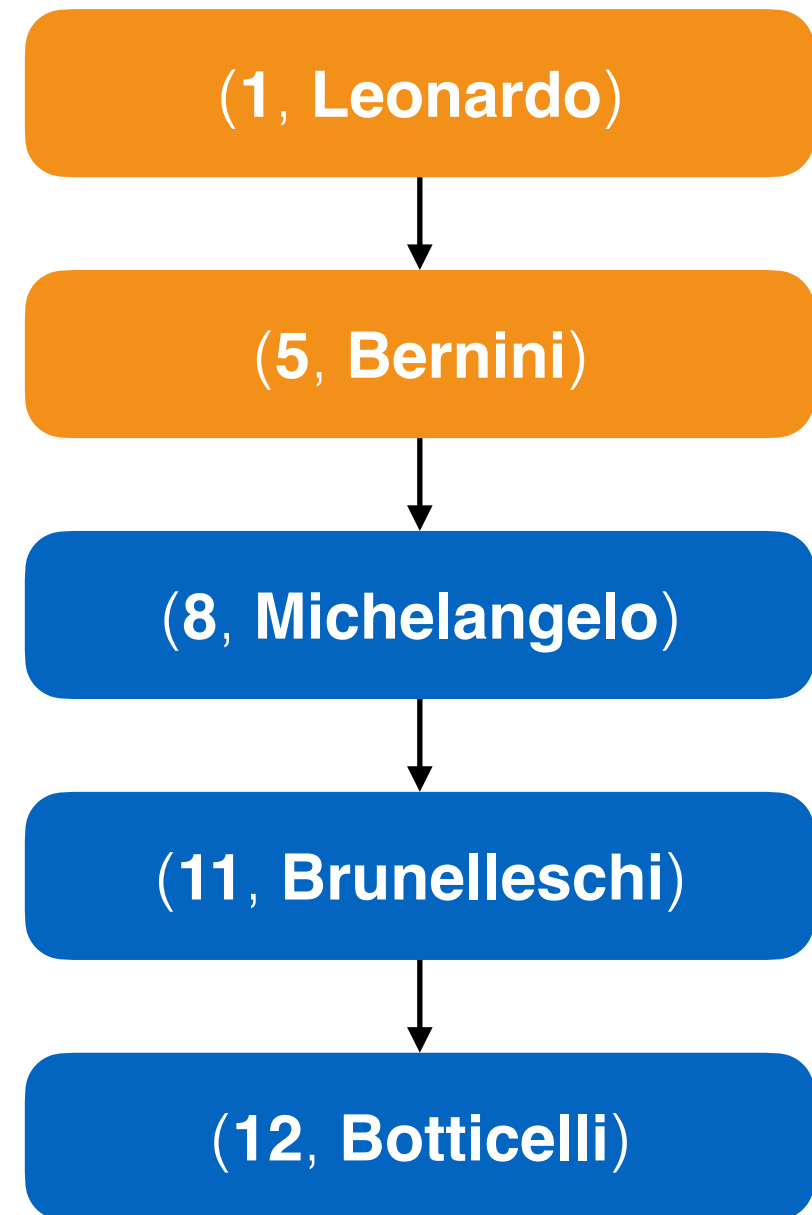
Supongamos que queremos el artista con identificador 11



Repaso (o spoiler) de EDD

Lista ligada (versión básica)

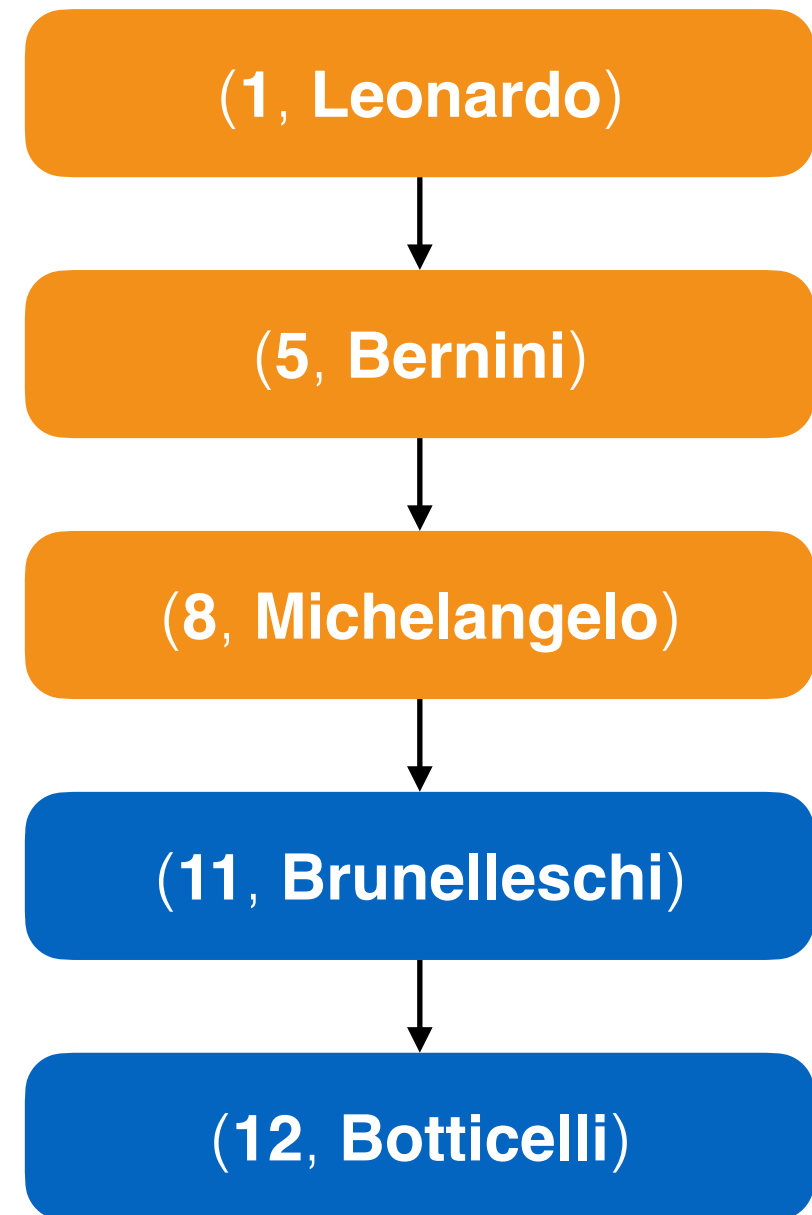
Supongamos que queremos el artista con identificador 11



Repaso (o spoiler) de EDD

Lista ligada (versión básica)

Supongamos que queremos el artista con identificador 11

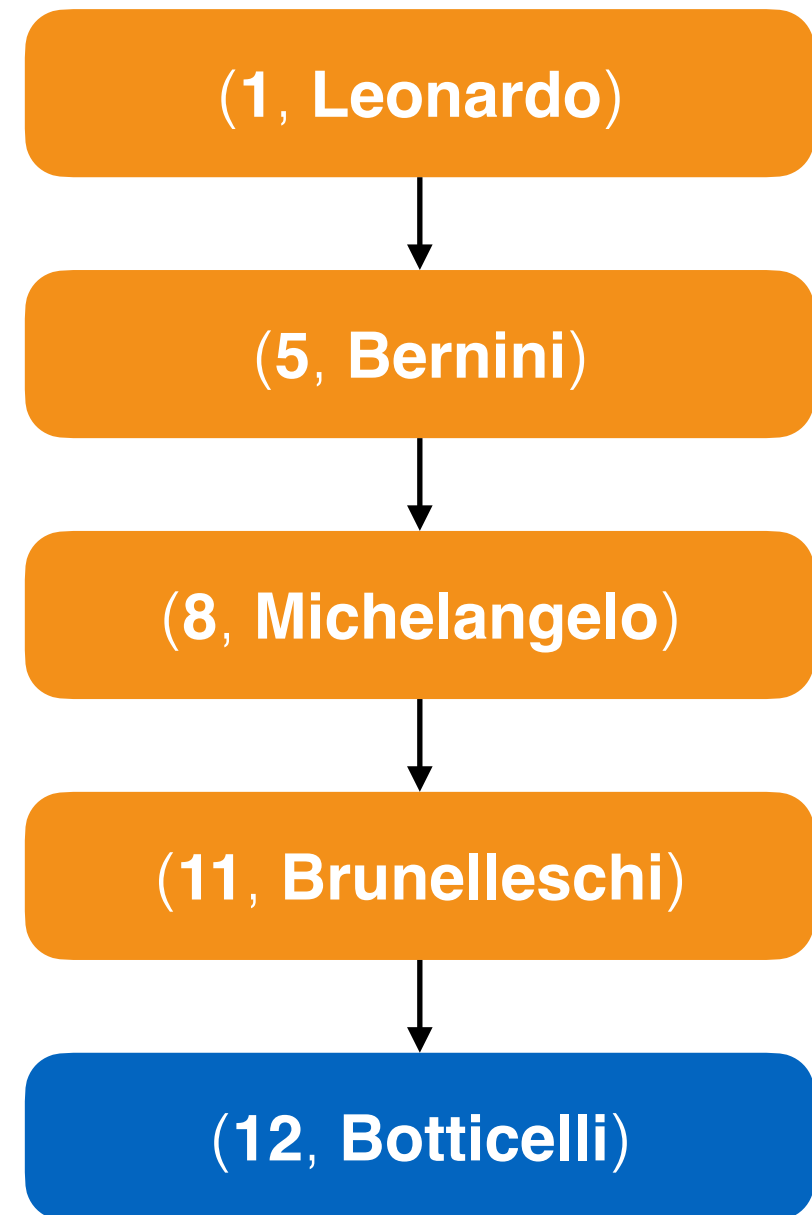


Repaso (o spoiler) de EDD

Lista ligada (versión básica)

Supongamos que queremos el artista con identificador 11

Buscar la tupla requerida toma **4** pasos



Repaso (o spoiler) de EDD

Tablas de Hash

Iniciamos un número fijo de casilleros (buckets)

En este caso cada tupla va a parar a un casillero según su **id**

Para esto utilizamos una función de **hash**

Repaso (o spoiler) de EDD

Tablas de Hash

La función de **hash** recibe el id del artista y retorna un número de casillero

Si tenemos 5 casilleros, una posible función es módulo 5 (retorna el resto de la división por 5)

Repaso (o spoiler) de EDD

Tablas de Hash



Función hash:
módulo 5 ($aid \% 5$)

Bucket	
0	
1	
2	
3	
4	

Repaso (o spoiler) de EDD

Tablas de Hash

(1, Leonardo)



Función hash:
módulo 5 ($aid \% 5$)

Bucket	
0	
1	
2	
3	
4	

Repaso (o spoiler) de EDD

Tablas de Hash

(5, Bernini)



Función hash:
módulo 5 ($aid \% 5$)

Bucket	
0	
1	(1, Leonardo)
2	
3	
4	

Repaso (o spoiler) de EDD

Tablas de Hash



Función hash:
módulo 5 ($aid \% 5$)

Bucket	
0	(5, Bernini)
1	(1, Leonardo)
2	
3	
4	

Repaso (o spoiler) de EDD

Tablas de Hash

(8, Michelangelo)

(11, Brunelleschi)

(12, Botticelli)



Función hash:
módulo 5 ($aid \% 5$)

Bucket

0

(5, Bernini)

1

(1, Leonardo)

2

3

4

Repaso (o spoiler) de EDD

Tablas de Hash



Función hash:
módulo 5 ($aid \% 5$)

Bucket	
0	(5, Bernini)
1	(1, Leonardo) (11, Brunelleschi)
2	(12, Botticelli)
3	(8, Michelangelo)
4	

Repaso (o spoiler) de EDD

Tablas de Hash

Si queremos buscar el artista con identificador 12, ¿cómo lo hacemos ahora? ¿en cuántos pasos lo obtenemos?

Repaso (o spoiler) de EDD

Tablas de Hash

aid = 12



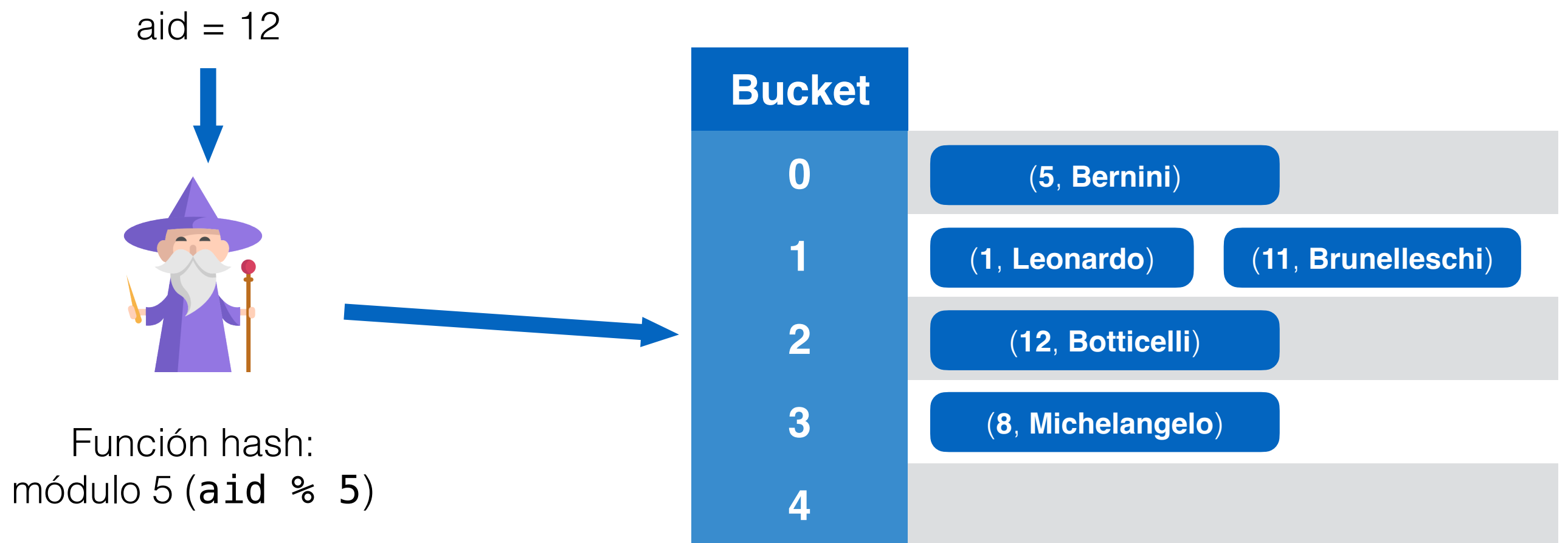
Función hash:
módulo 5 ($\text{aid} \% 5$)

Bucket	
0	(5, Bernini)
1	(1, Leonardo) (11, Brunelleschi)
2	(12, Botticelli)
3	(8, Michelangelo)
4	

Computamos el hash del número 12

Repaso (o spoiler) de EDD

Tablas de Hash



Y en 1 paso encontramos el casillero que le corresponde

Repaso (o spoiler) de EDD

Tablas de Hash

Notamos que hay colisiones, esto es, dos valores que van a parar al mismo casillero

En general, suponemos que las funciones de hash distribuyen uniforme

También suponemos que hay suficientes casilleros para que la búsqueda se haga en una cantidad de pasos cercano a 1

Hash Index

La idea es replicar el concepto de las tablas de hash pero en un sistema de bases de datos

Aquí nuestros casilleros serán páginas del disco duro

Recordemos que en cada página caben muchas tuplas

Hash Index

Recordemos que queremos encontrar tuplas para consultas de igualdad de forma rápida

```
SELECT * FROM Artistas A WHERE A.aid = 12
```

Y que rápido significa hacer la menor cantidad de I/O posible

Hash Index

Ejemplo

Vamos a retomar el ejemplo de los artistas, pero ahora insertando las tuplas a una base de datos

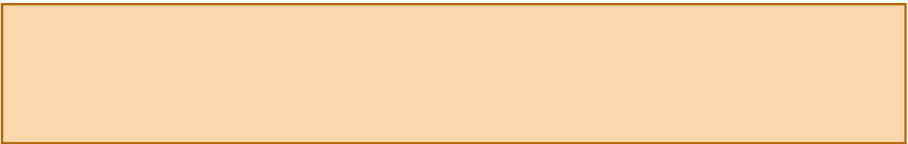
Artista.aid	Artista.nombre
1	Leonardo
5	Bernini
8	Michelangelo
11	Brunelleschi
12	Botticelli
16	Giotto

Hash Index

Ejemplo

Artista.aid	Artista.nombre
1	Leonardo
5	Bernini
8	Michelangelo
11	Brunelleschi
12	Botticelli
16	Giotto

0



1



2



3



4

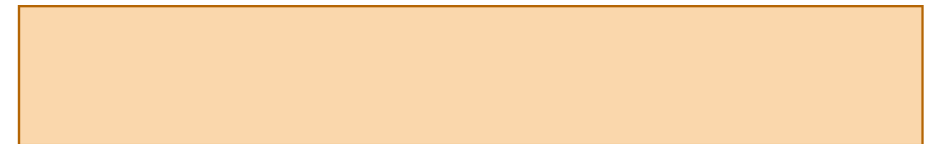


Hash Index

Ejemplo

Artista.aid	Artista.nombre
1	Leonardo
5	Bernini
8	Michelangelo
11	Brunelleschi
12	Botticelli
16	Giotto

0



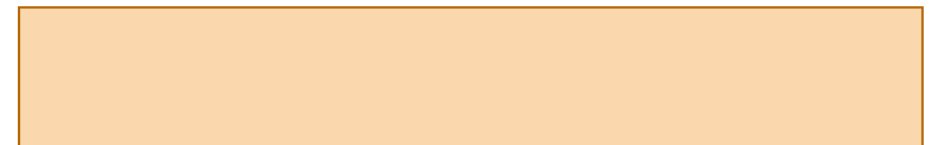
1



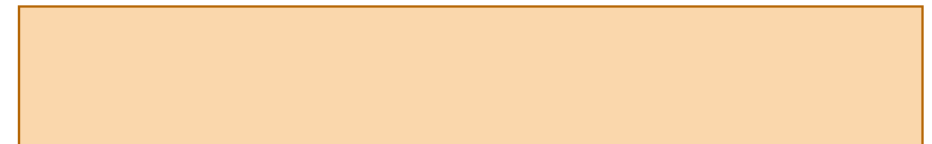
2



3



4



Hash Index

Ejemplo

Artista.aid	Artista.nombre
1	Leonardo
5	Bernini
8	Michelangelo
11	Brunelleschi
12	Botticelli
16	Giotto

0

(5, Bernini)

1

(1, Leonardo)

2

3

4

Hash Index

Ejemplo

Artista.aid	Artista.nombre
1	Leonardo
5	Bernini
8	Michelangelo
11	Brunelleschi
12	Botticelli
16	Giotto

0

(5, Bernini)

1

(1, Leonardo)

2

3

(8, Michelangelo)

4

Hash Index

Ejemplo

Artista.aid	Artista.nombre
1	Leonardo
5	Bernini
8	Michelangelo
11	Brunelleschi
12	Botticelli
16	Giotto

0

(5, Bernini)

1

(1, Leonardo)

(11, Brunelleschi)

2

3

(8, Michelangelo)

4

Hash Index

Ejemplo

Artista.aid	Artista.nombre
1	Leonardo
5	Bernini
8	Michelangelo
11	Brunelleschi
12	Botticelli
16	Giotto

0

(5, Bernini)

1

(1, Leonardo)

(11, Brunelleschi)

2

(12, Botticelli)

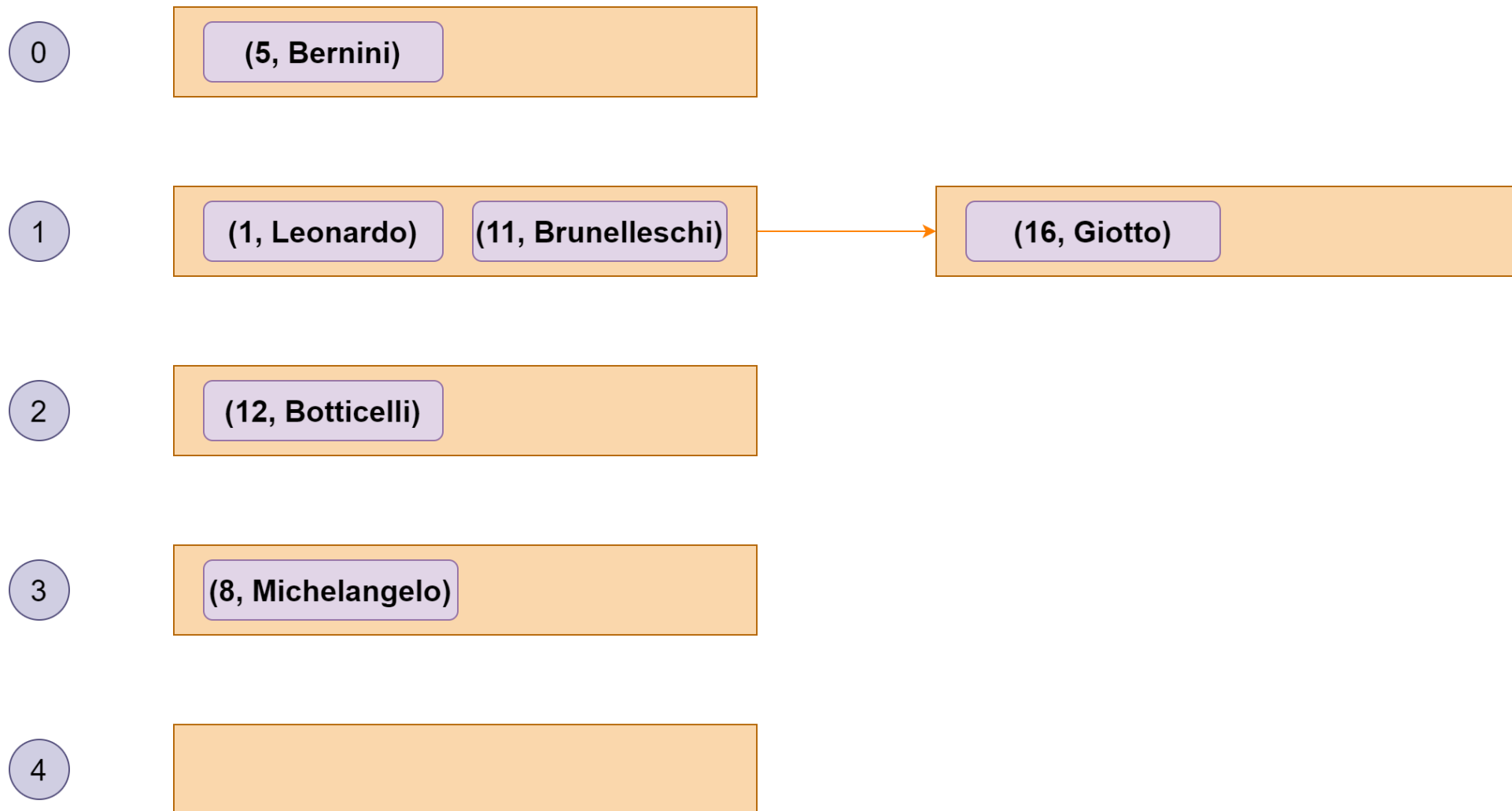
3

(8, Michelangelo)

4

Hash Index

Ejemplo



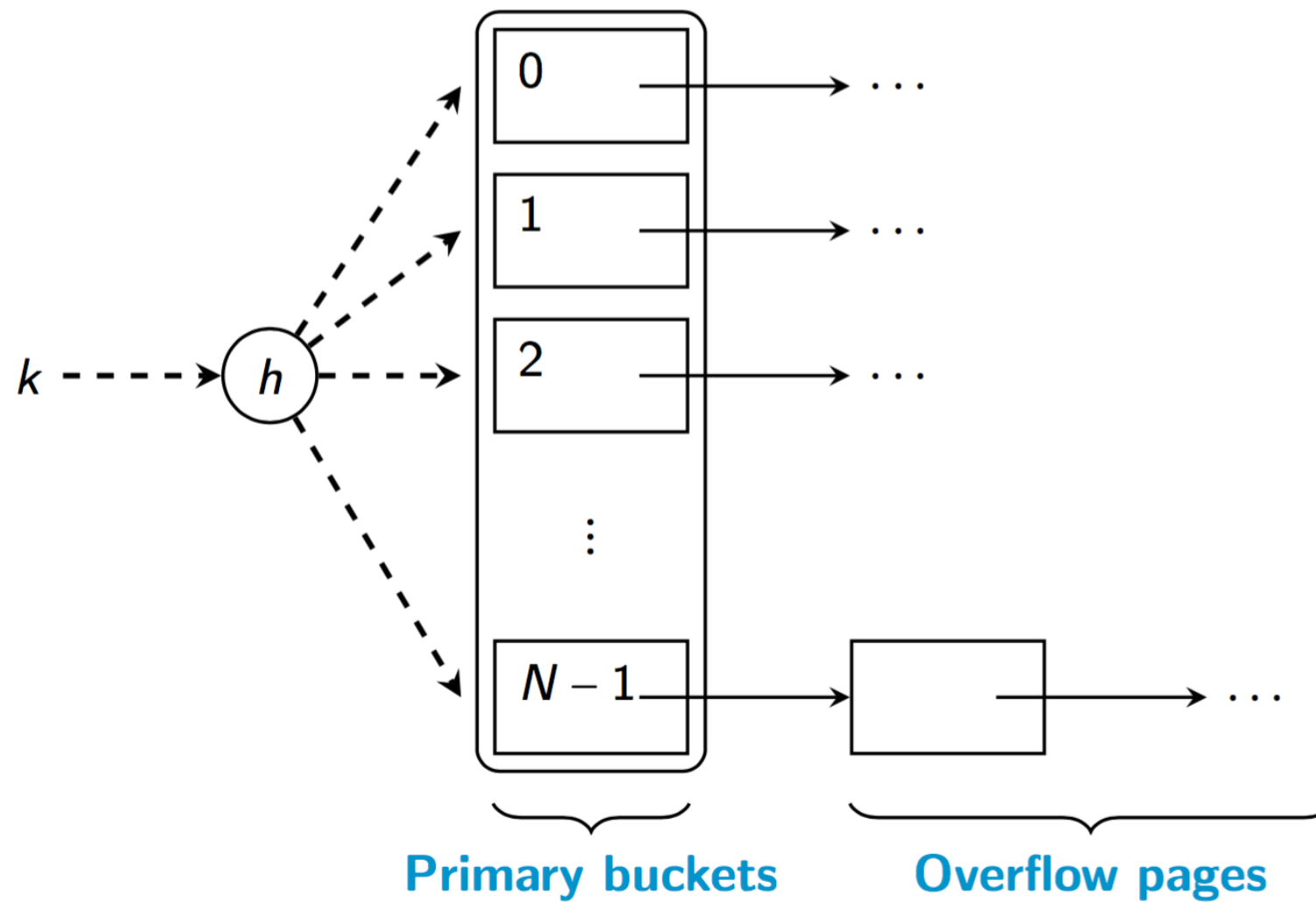
Hash Index

Para una relación **R** y un atributo **A**

- N páginas de disco sirven de **buckets**
- Cada bucket cuenta con una lista ligada de **overflow pages**
- Usamos una función de hash **h** que me indicará que bucket le corresponde a cada valor:

$$h: \text{dominio}(k) \rightarrow [0, \dots, n - 1]$$

Hash Index



Hash Index

Para buscar un elemento dada una search key **k**:

- Se computa el valor de **$h(k)$**
- Se accede al bucket correspondiente
- Se busca el elemento en la página asociada o una de las overflow pages

Hash Index

¿Y qué pasa con las colisiones?

Hash Index

$$\text{Costo del Hash Index en I/O} \sim \frac{|(\textit{DataEntries})|}{B \cdot N}$$

B = Elementos que caben en una página

N = Número de buckets

|Data Entries| = Número de elementos guardados

¿Por qué?

Hash Index

$$\text{Costo del Hash Index en I/O} \sim \frac{|(\textit{DataEntries})|}{B \cdot N}$$

B = Elementos que caben en una página

N = Número de buckets

|Data Entries| = Número de elementos guardados

Para N grande, esto se asemeja a 1.
Pero cuando hay colisiones, puede aumentar.

Hash Index

¿Qué pasa si hay muchas colisiones?

Hash Index Dinámico

Existen tipos de Hash Index que crecen a medida que es necesario:

- Extendable Hash Index
- Linear Hash Index

En general, tienden a mantener un costo de búsqueda constante (en I/O) cercano a 1

Hash Index

¿Cuando **no** nos conviene utilizar un Hash Index?

Hash Index

¿Cuándo **no** nos conviene utilizar un Hash Index?

```
SELECT * FROM Artistas A  
WHERE aid >= 5 AND aid <= 20
```

No existe relación entre $h(5)$ y $h(6)$
(que corresponden a $aid = 5$ y $aid = 6$)

B+ Tree Index

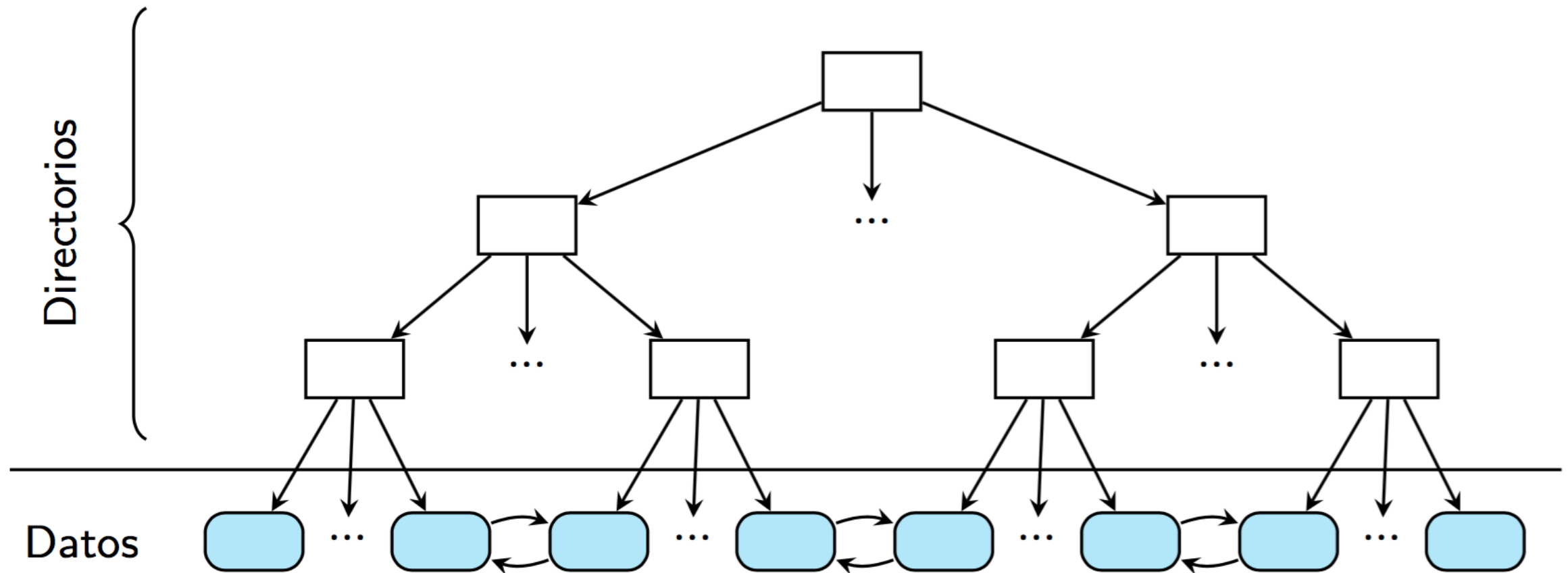
*“B+ Trees are by far the most important access path structure in database and file systems”,
Gray y Reuter (1993).*

B+ Tree Index

Es un índice basado en un árbol:

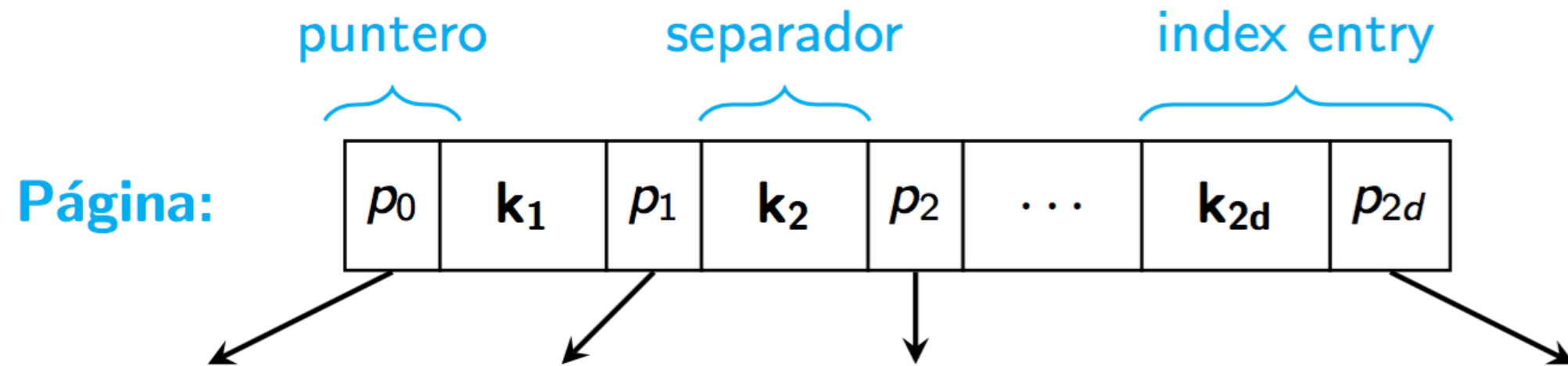
- Las páginas del disco representan nodos del árbol
- **Las tuplas** están en las hojas
- Provee un tiempo de búsqueda logarítmico
- Se comporta bien en consultas de rango
- Es el índice que usa Postgres sobre las llaves primarias (y en general, todos los sistemas)

B+ Tree Index



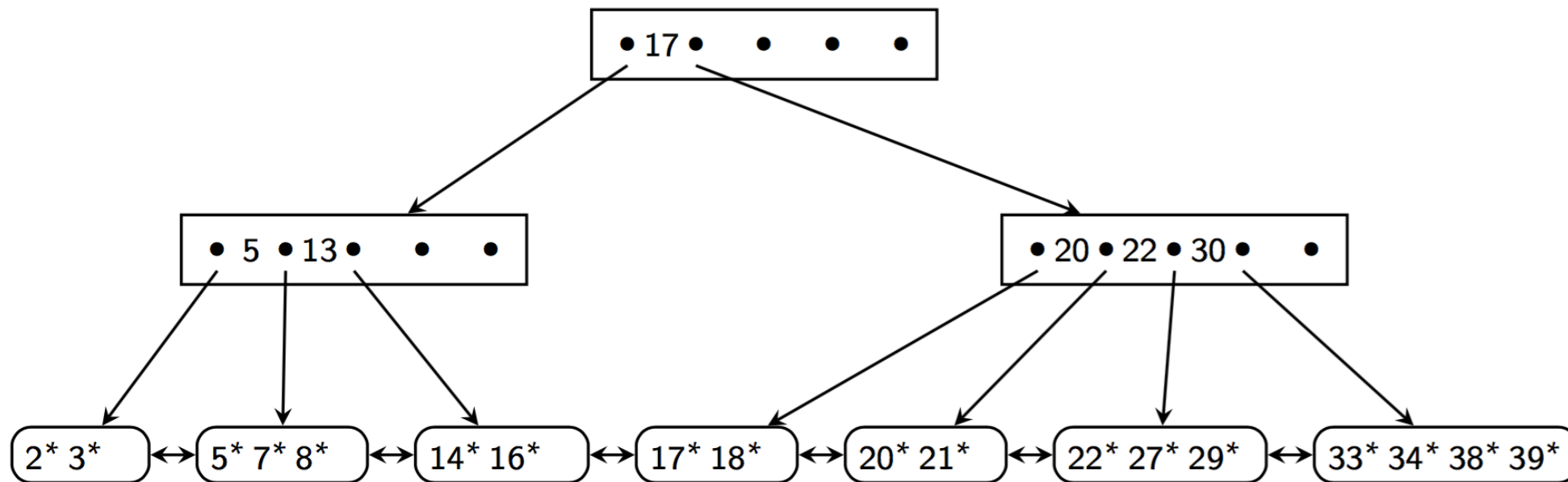
B+ Tree Index

Cada nodo tiene la siguiente forma:



B+ Tree Index

En los directorios, los nodos tienen una **Search Key** y un puntero antes y después de ella

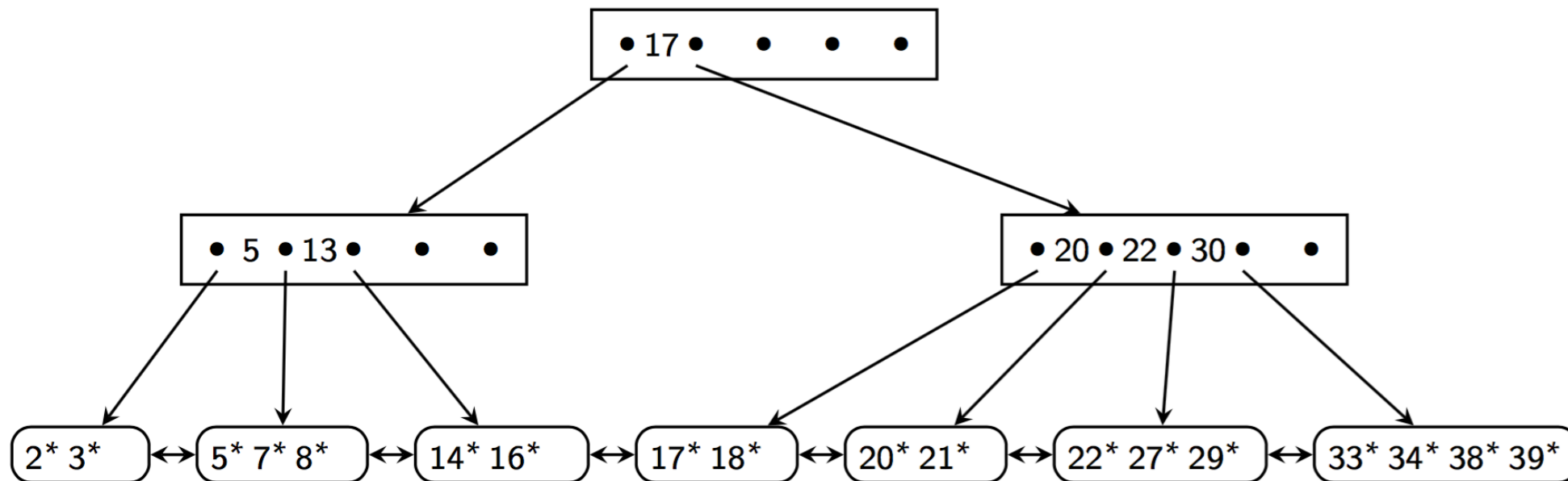


B+ Tree Index

Ejemplo: consulta por un valor

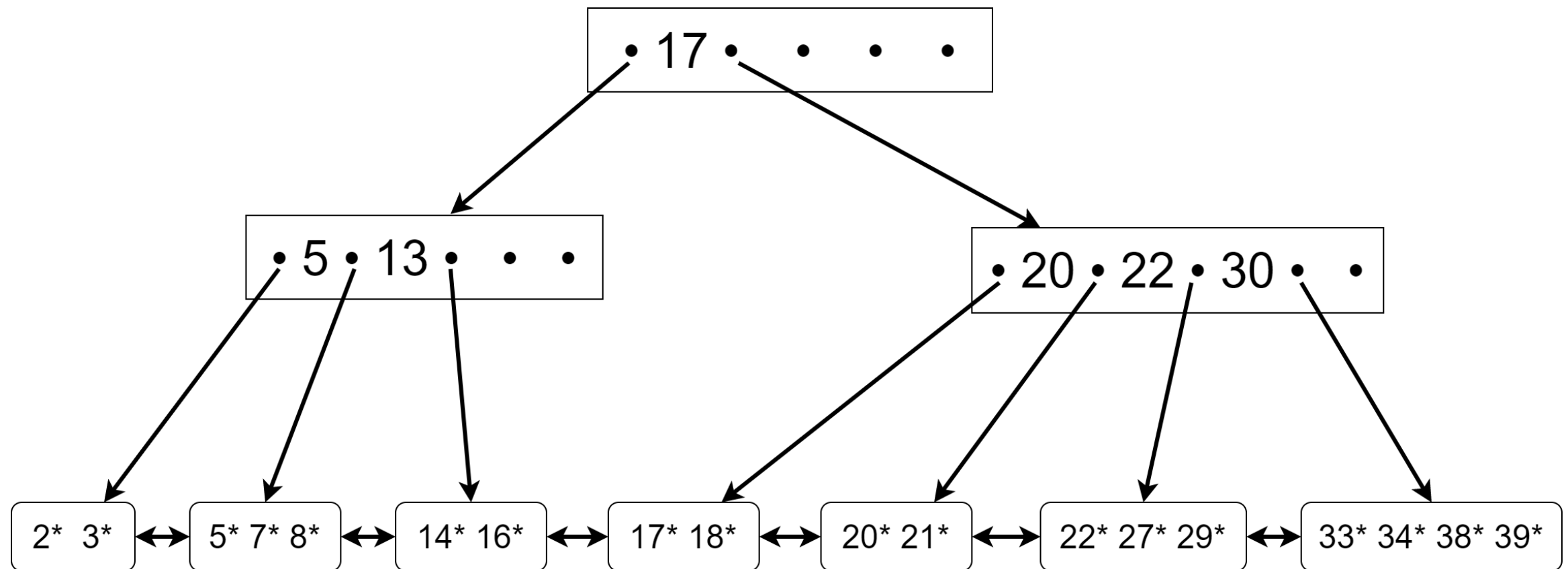
Supongamos que este índice también almacena artistas que están indexados por su **aid**

Queremos el artista con **aid** = 21



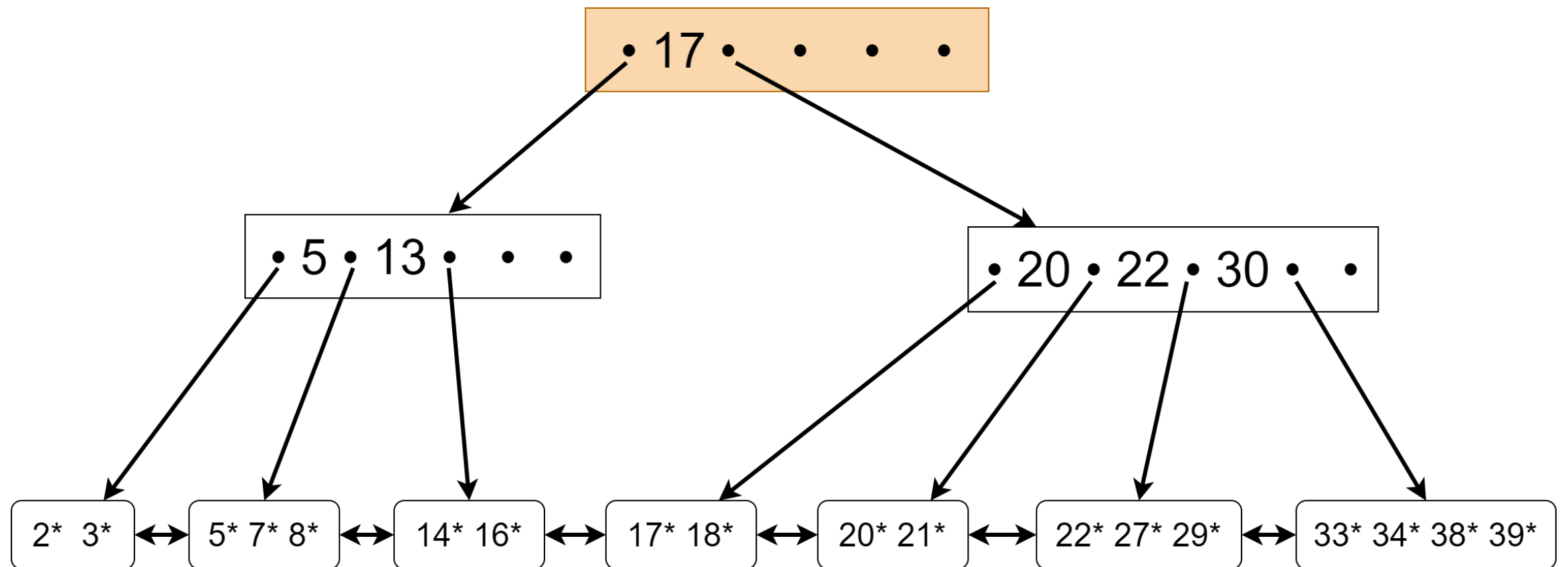
B+ Tree Index

Ejemplo: consulta por un valor



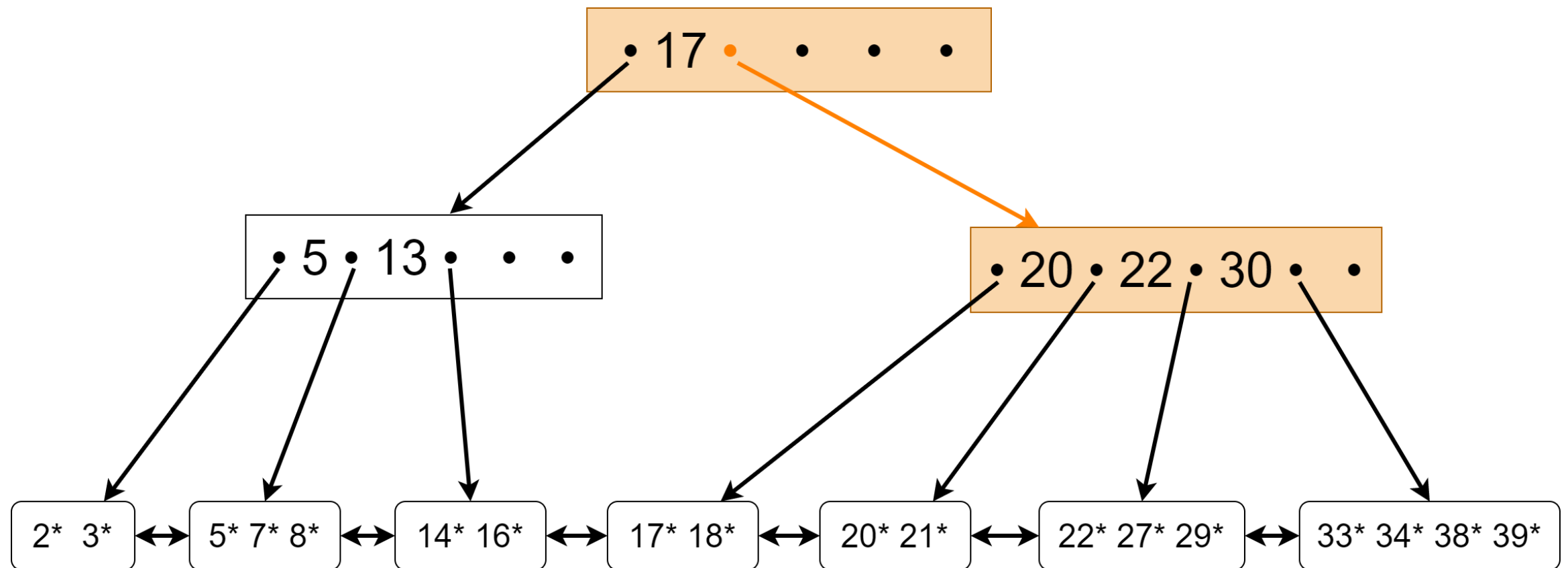
B+ Tree Index

Ejemplo: consulta por un valor



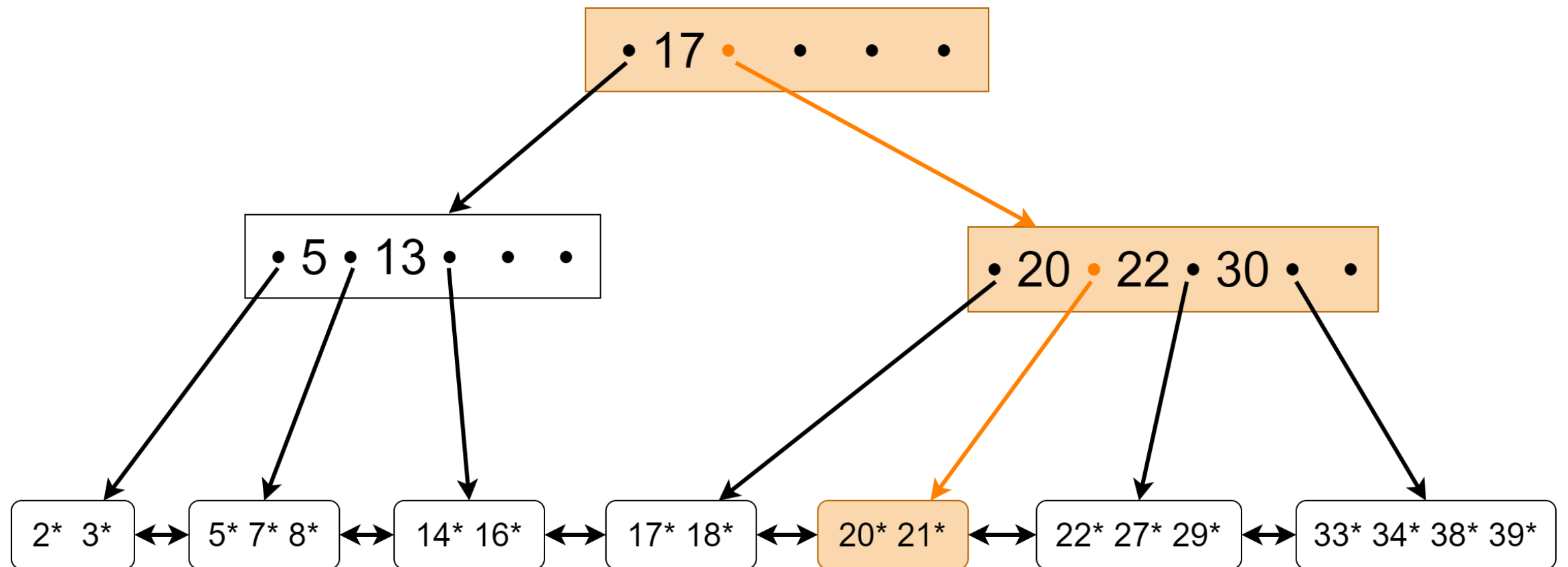
B+ Tree Index

Ejemplo: consulta por un valor



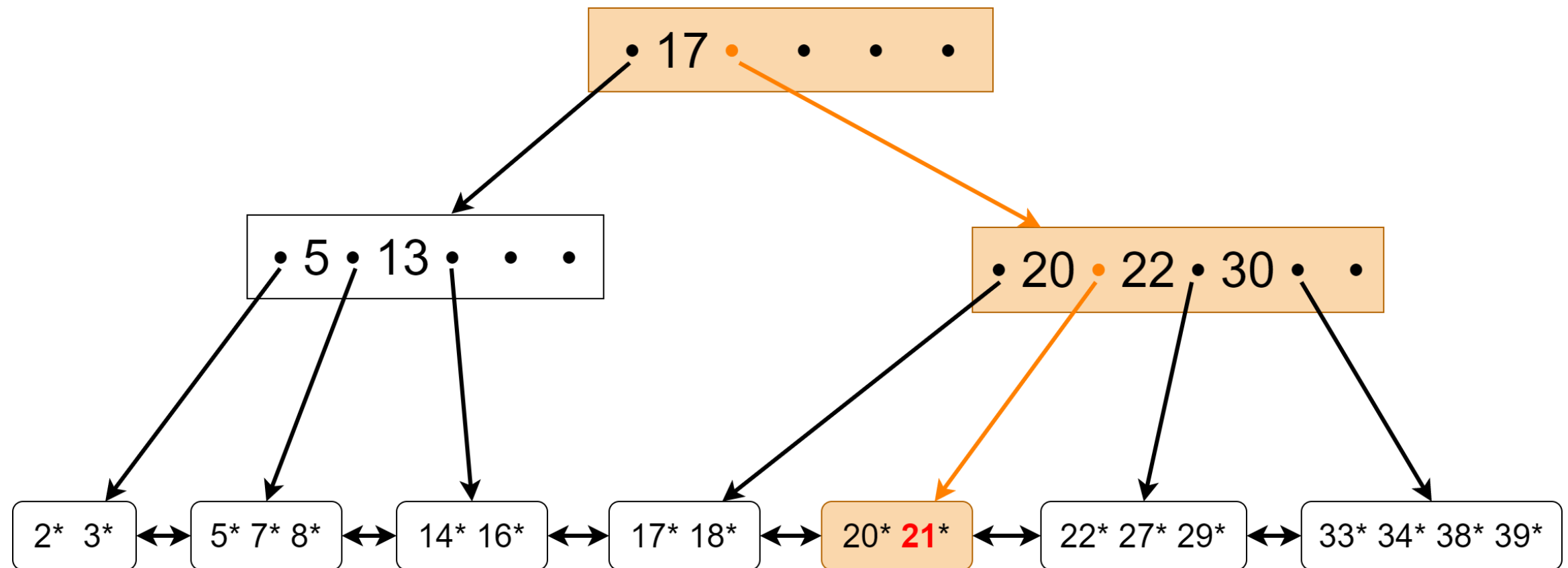
B+ Tree Index

Ejemplo: consulta por un valor



B+ Tree Index

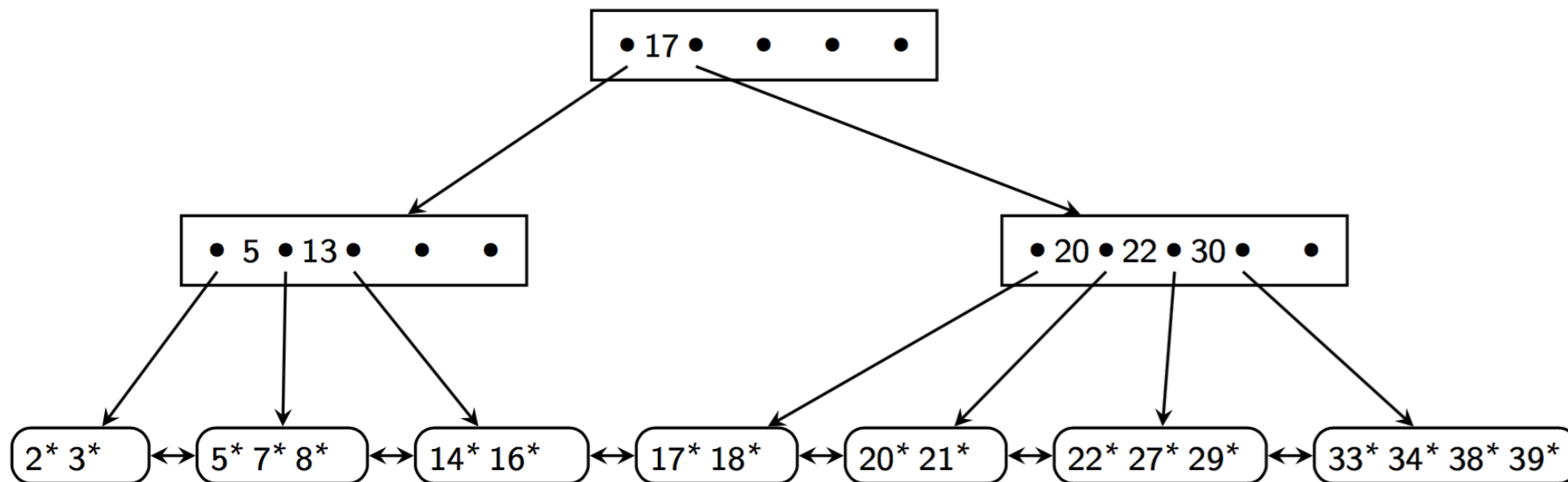
Ejemplo: consulta por un valor



¿Y por qué este índice me sirve
en consultas de rango?

B+ Tree Index

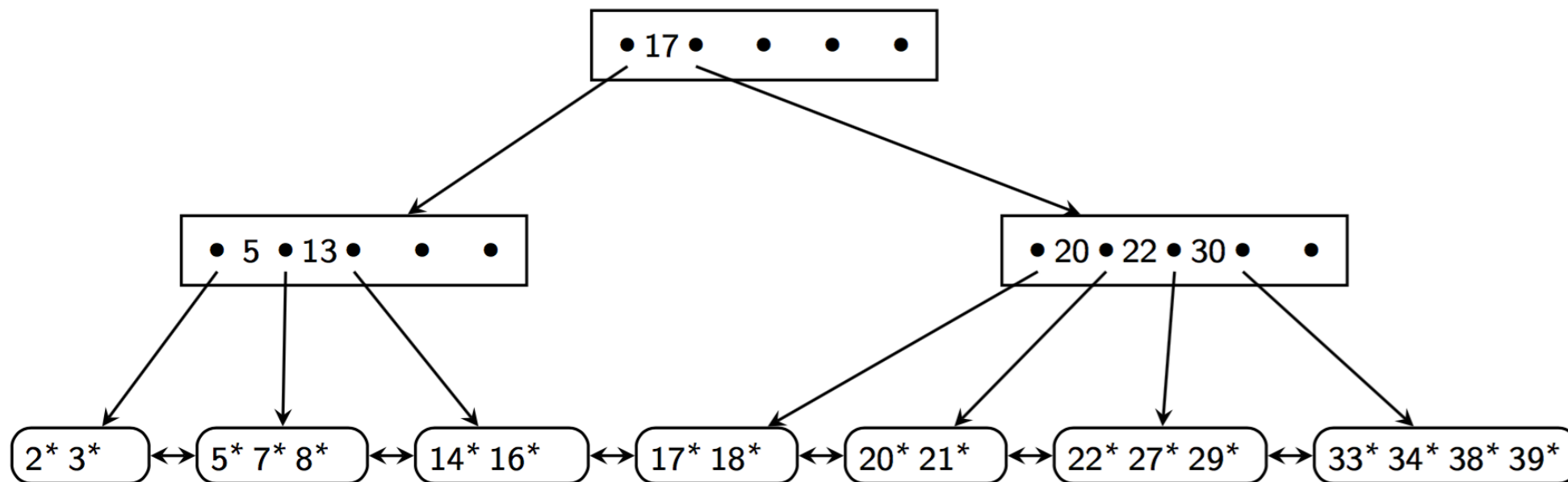
Sabemos que las tuplas van a estar ordenadas según el atributo indexado, y además las hojas están encadenadas unas con otras



B+ Tree Index

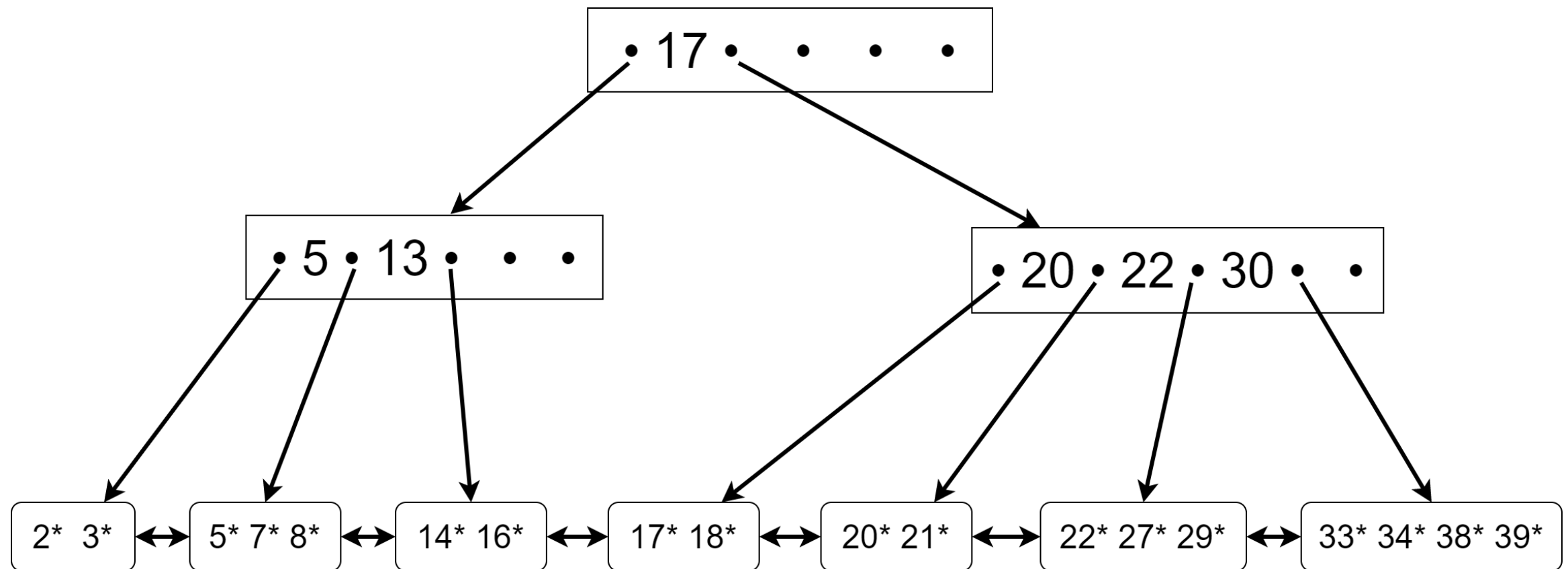
Ejemplo: consulta de rango

Ahora queremos hacer la consulta de todos los Artistas con **aid** entre 5 y 26



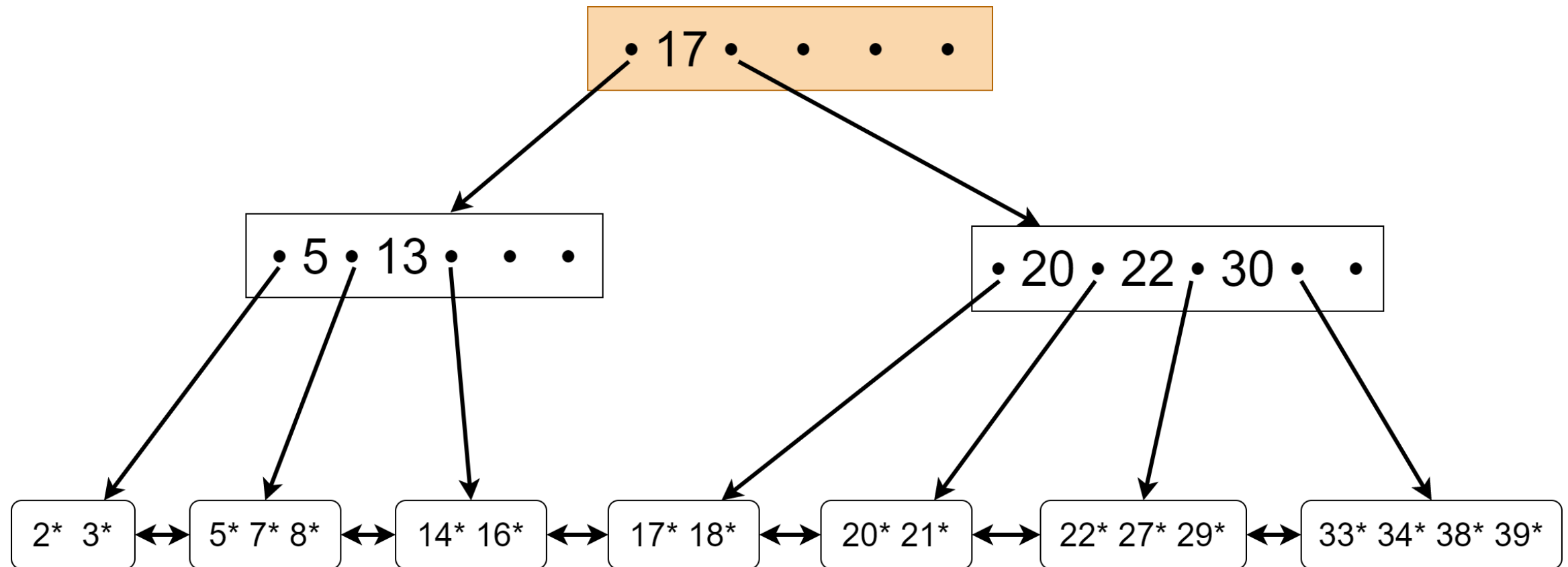
B+ Tree Index

Ejemplo: consulta de rango



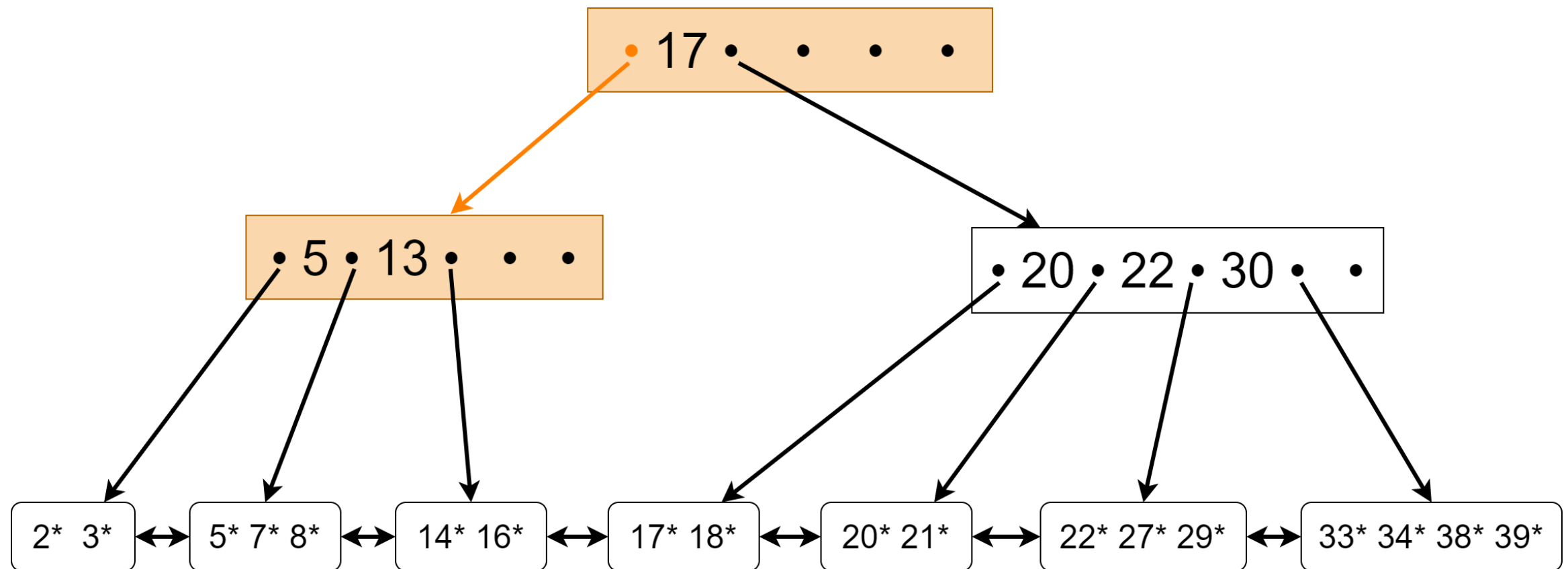
B+ Tree Index

Ejemplo: consulta de rango



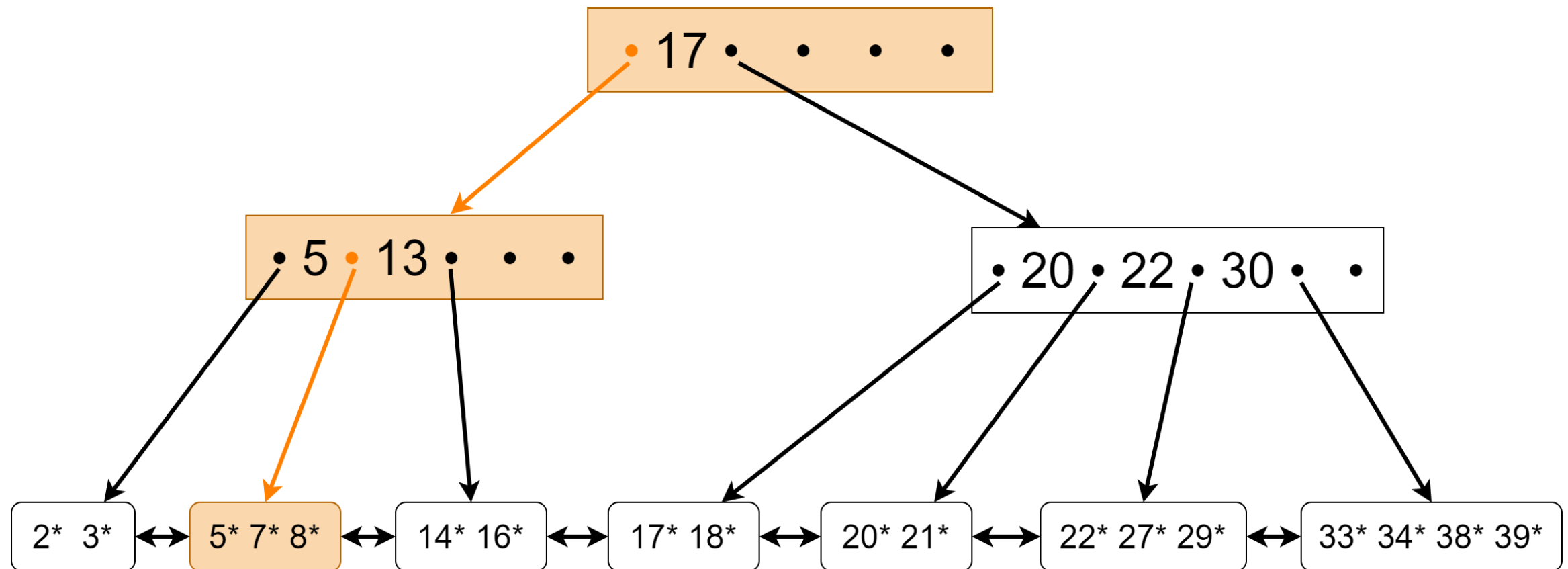
B+ Tree Index

Ejemplo: consulta de rango



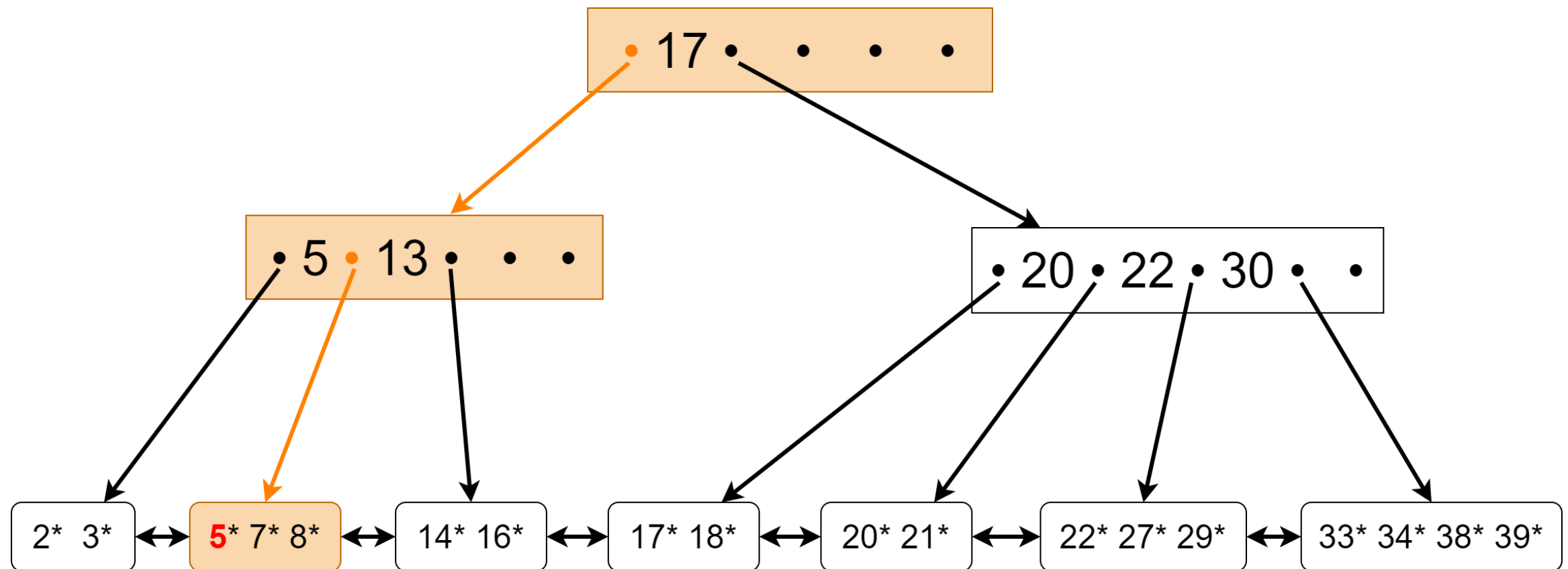
B+ Tree Index

Ejemplo: consulta de rango



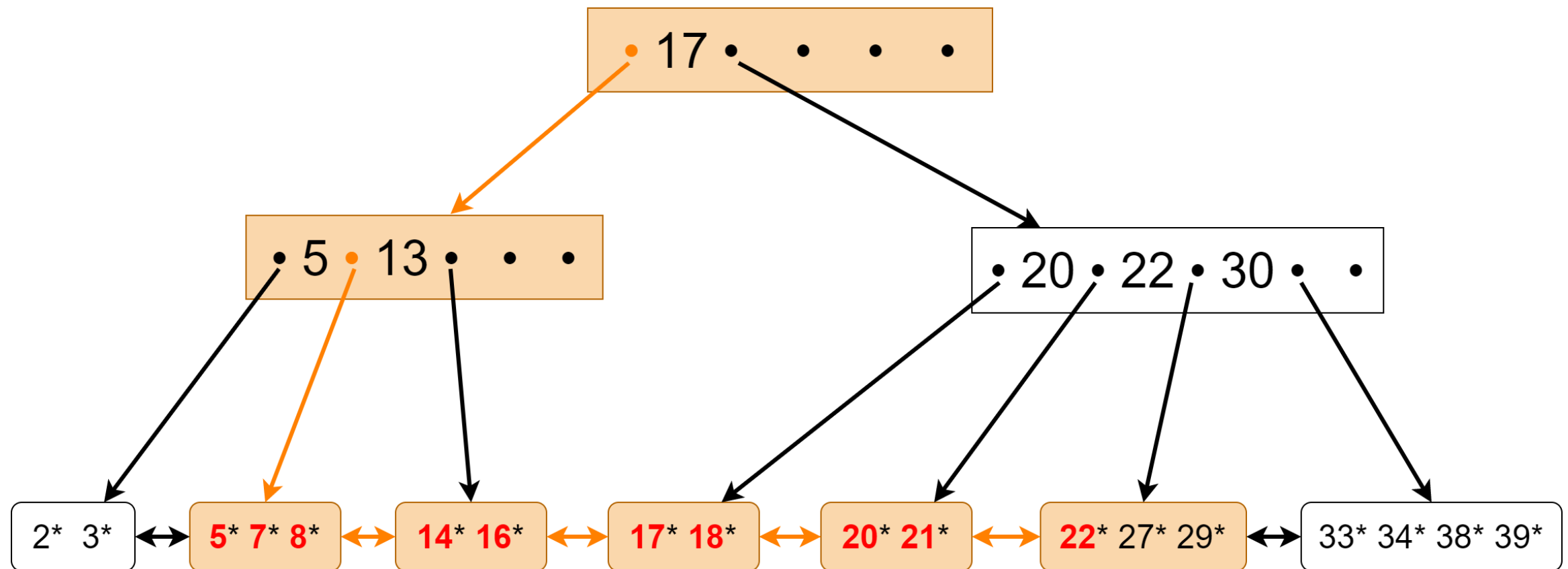
B+ Tree Index

Ejemplo: consulta de rango



B+ Tree Index

Ejemplo: consulta de rango



B+ Tree Index

Todos los nodos del B+Tree deben estar utilizados a más de la mitad de su capacidad, excepto la raíz, que puede tener menos elementos

B+ Tree Index

Al insertar, eliminar o actualizar valores debemos mantener el árbol balanceado, cumpliendo con mantener el orden del árbol

Esto último mantiene el tiempo de búsqueda logarítmico

B+ Tree Index

El costo de búsqueda es logarítmico

Si **B** es el tamaño de una página en tuplas, el costo de búsqueda es:

$$O(\log_{\frac{B}{2}}(\frac{\text{Tuplas}}{\frac{B}{2}})) = O(\log_{\frac{B}{2}}(\frac{2 \cdot \text{Tuplas}}{B}))$$

B+ Tree Index

El costo de búsqueda es logarítmico

Si **B** es el tamaño de una página en tuplas, el costo de búsqueda es:

$$O(\log_{\frac{B}{2}}(\frac{\text{Tuplas}}{\frac{B}{2}})) = O(\log_{\frac{B}{2}}(\frac{2 \cdot \text{Tuplas}}{B}))$$

En la práctica esto es menor por que los niveles altos del árbol se guardan en RAM

Hash vs B+ Tree

Criterio	Hash Index	B+ Tree
Consulta de igualdad	≈ 1 I/O (óptimo)	$O(\log N)$ I/O
Consulta de rango	No sirve	Excelente
Mantenimiento	Puede requerir rehash	Split/merge automático
Uso en PostgreSQL	Solo con <code>CREATE INDEX USING hash</code>	Por defecto en PK

Como B+ Tree sirve para igualdad y rango, se usa por defecto.

Vamos a ver esto más adelante, con algoritmos.

Tipos de valores a guardar

Un índice me puede retornar:

- La tupla que estoy buscando
- Un puntero o lista de punteros a las páginas del disco duro que tienen las tuplas que estoy buscando

Clustered y Unclustered Index

Hablaremos de un índice **Clustered** si al preguntarle por un valor me retorna una tupla

Hablaremos de un índice **Unclustered** si me retorna un puntero

Índice Clustered

Una (no tan importante) Aclaración

En realidad la definición de un **Clustered Index** señala que es un índice que está ordenado de la misma forma que los records en la DB

Y un **Clustered File** es un índice que me retornará la tupla en vez de un puntero

Índice Clustered

Una (no tan importante) Aclaración

Rara vez encontraremos un Clustered Index que no sea un Clustered File, por lo que para efectos de este curso consideraremos que la definición de Clustered Index es la que dimos dos diapositivas atrás

Clustered y Unclustered Index

Clustered Index se conoce como índice primario

Unclustered Index se conoce como índice secundario

Clustered y Unclustered Index

Considere Empleados(id, nombre, edad)

Un índice primario en general se aplica sobre la primary key, por ejemplo el id de los empleados

Un índice secundario se aplica sobre un atributo como edad (para ordenar por edad más rápido)

Clustered vs Unclustered

```
SELECT * FROM Empleados WHERE edad BETWEEN 25 AND 30
```

Con clustered B+ Tree index en edad:

- tuplas contiguas en disco, lectura secuencial
- Costo = profundidad + páginas en rango

Con unclustered B+ Tree index en edad:

- Para acceder a tuplas necesitamos accesos aleatorios
- Costo peor caso = profundidad + 1 random scan por tupla!

Diseño de índices

Más índices:

- Acceso más rápido
- Pero más memoria
- Actualización más lenta

¿Hasta cuándo indexar?

No hay una respuesta fácil, para responder bien hay que monitorear el rendimiento y conocer los requerimientos de tiempo / espacio.

Guía práctica

Empleados(id, nombre, depto, salario)

Consulta frecuente	Índice recomendado
WHERE id = 104	B+ Tree en id (ya existe: PK)
WHERE depto = 'Ventas'	B+ Tree o Hash en depto
WHERE salario > 50000	B+ Tree en salario (rango)
WHERE depto = 'Ventas' AND salario > 50000	B+ Tree compuesto en (depto, salario)

Diseño de índices

Índices compuestos:

- Search key con varios atributos
- El orden importa (depto, salario) filtra primero por adepto, luego salario.
- (Salario, depto) filtra primero por salario, luego por depto.